

CEIS10 PROGRAMMING WITH DATA

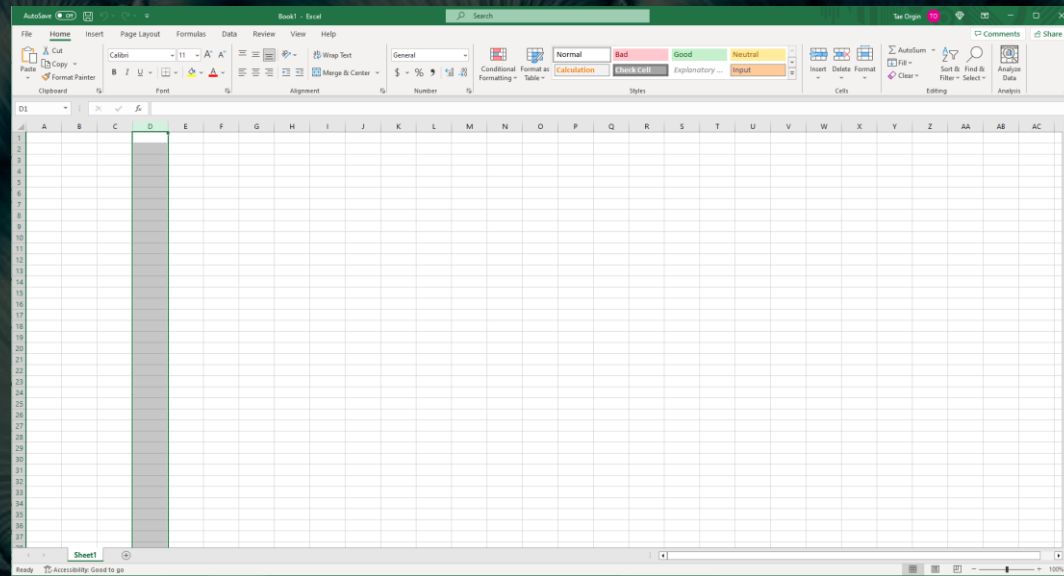
By: Deontae Crenshaw

INTRODUCTION

- Data is growing exponentially
- This project uses a cloud-based system to gather temperature and humidity data
- Then the data is analyzed using programming and data analytics

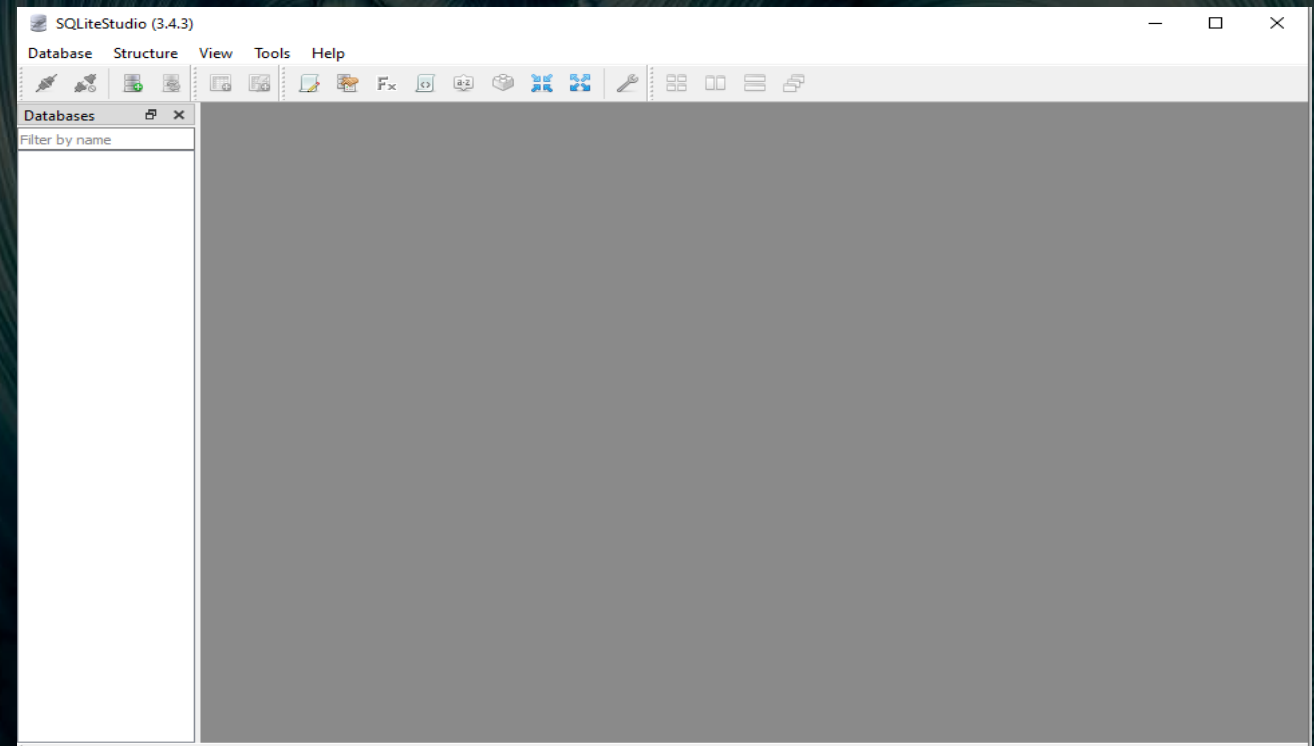
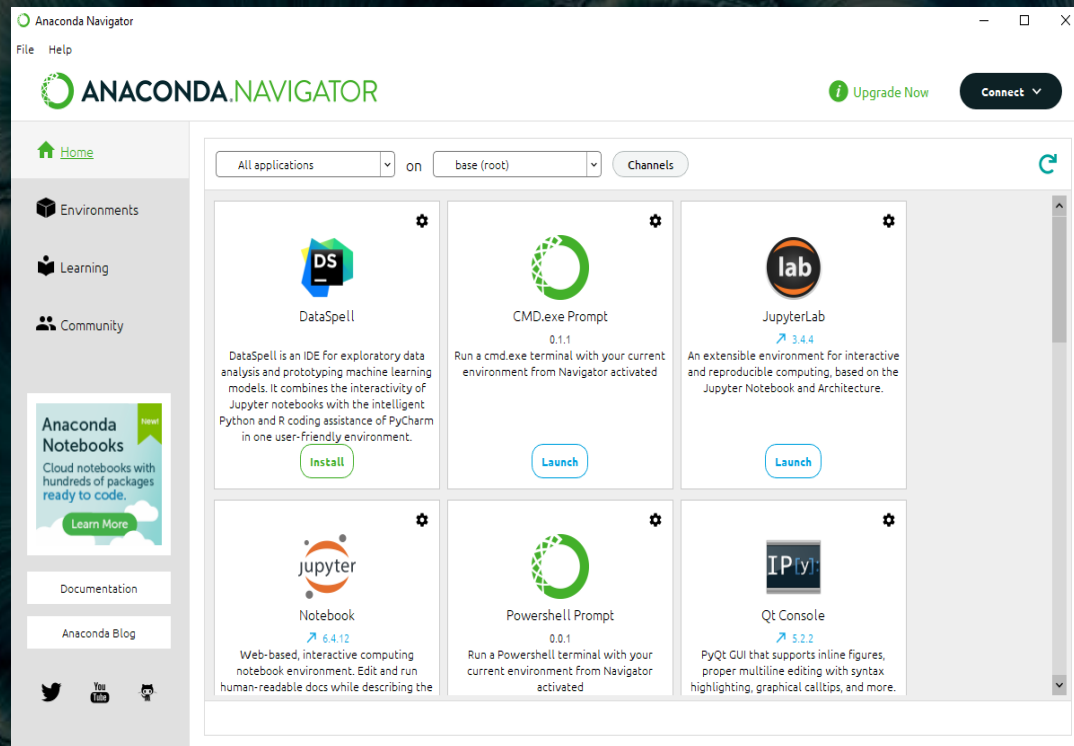
SOFTWARE INVENTORY

- Before developing this programming with data project, all software must be downloaded and installed.
- Software needed included SQLite studio, Excel, and a python IDE - anaconda



SOFTWARE

The software needed for this project includes:
Microsoft Excel
Anaconda W/ Spyder
SQLite



PLANNING AND DESIGN

- After performing an inventory of the software needed for the project, a plan was created for the temperature data project.
- Created a flowchart

WHAT ARE FLOWCHARTS?

Flowcharts are graphical representations of processes or workflows, and the flowchart developed illustrates the process and the output of this software development project.

Start

Install Python Modules

Download weather to a database

Extract weather data to a csv file

Cleanse weather data

Use excel to manipulate data

Use python data analytics modules to develop
graphical modules

End

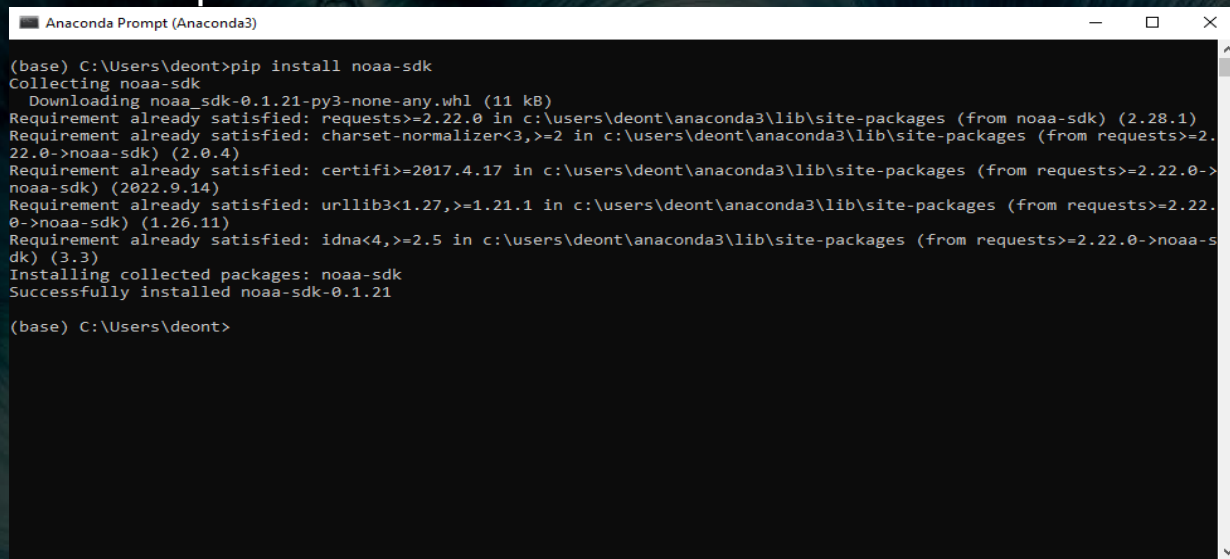
Flowchart

Include the following processes:

- Install python
- Download weather data to a database.
- Extract weather data from database into a comma separated file with python
- Cleanse weather data
- Use Excel to manipulate data
- Use python data analytics modules to develop graphical models

ADDING LIBRARY

- Connected python to the U.S. government National Oceanic and Atmospheric Administration.



```
(base) C:\Users\deont>pip install noaa-sdk
Collecting noaa-sdk
  Downloading noaa_sdk-0.1.21-py3-none-any.whl (11 kB)
Requirement already satisfied: requests>=2.22.0 in c:\users\deont\anaconda3\lib\site-packages (from noaa-sdk) (2.28.1)
Requirement already satisfied: charset-normalizer<3,>=2 in c:\users\deont\anaconda3\lib\site-packages (from requests>=2.22.0->noaa-sdk) (2.0.4)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\deont\anaconda3\lib\site-packages (from requests>=2.22.0->noaa-sdk) (2022.9.14)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in c:\users\deont\anaconda3\lib\site-packages (from requests>=2.22.0->noaa-sdk) (1.26.11)
Requirement already satisfied: idna<4,>=2.5 in c:\users\deont\anaconda3\lib\site-packages (from requests>=2.22.0->noaa-sdk) (3.3)
Installing collected packages: noaa-sdk
Successfully installed noaa-sdk-0.1.21

(base) C:\Users\deont>
```

- The screen shows that the NOAA-SDK library is installed

GATHERING TEMPERATURE AND HUMIDITY DATA

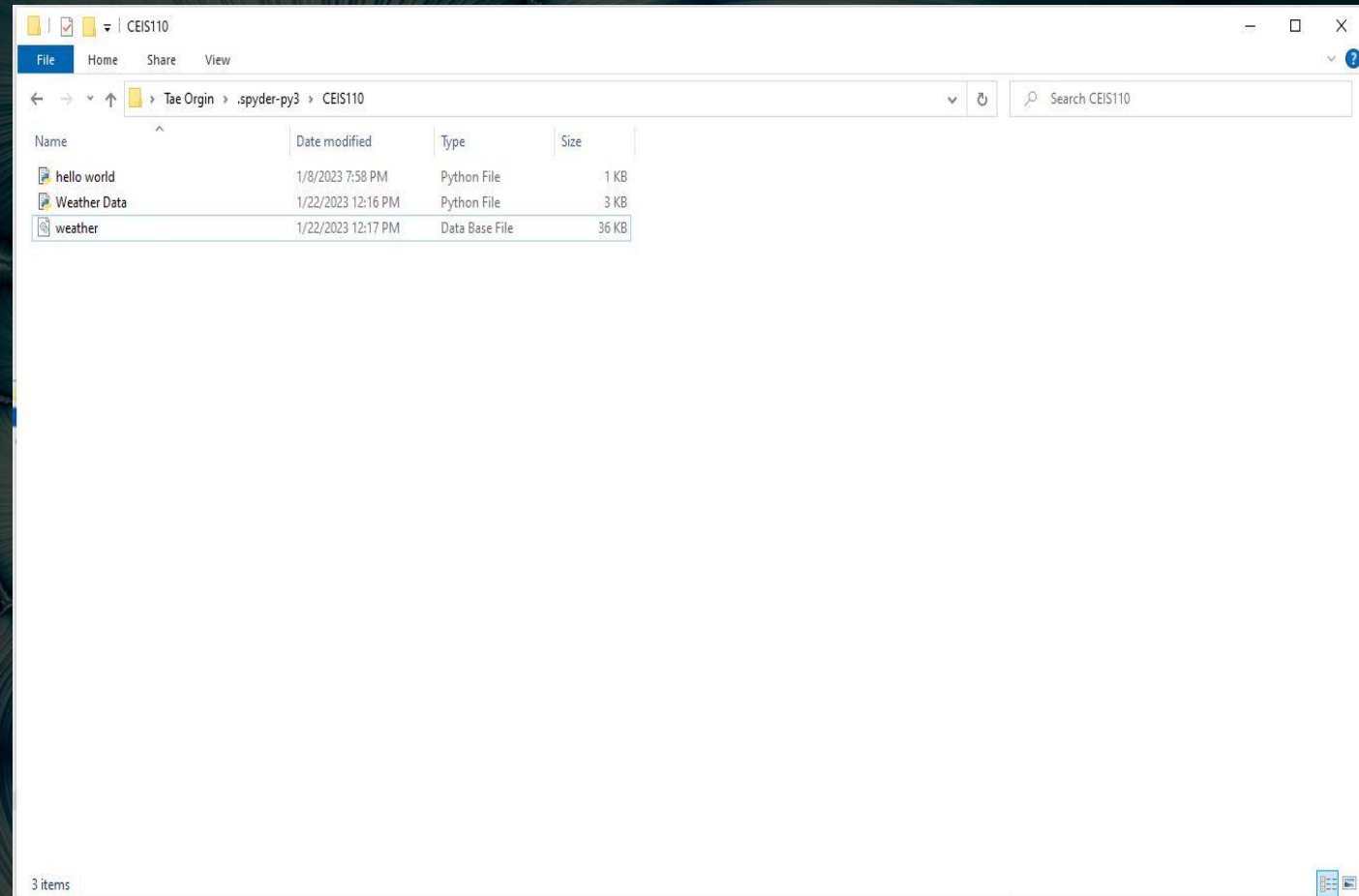
The code was developed to download a set of weather observations, data was stored on a local database later for analysis

BUILDWEATHERDB.PY

The screenshot displays the Spyder Python IDE interface. The main editor shows the script `Weather Data.py`, which is designed to build a weather database from NOAA data. The script includes comments, imports for `noaa_sdk`, `sqlite3`, and `datetime`, and defines parameters for retrieving NOAA weather data. It then creates a SQLite database, drops any existing table, and creates a new table to store observations. Finally, it fetches hourly weather observations from the NOAA Weather Service API and inserts them into the database.

The console window at the bottom shows the execution output of the script. It indicates that the database was prepared, weather data was fetched, and 225 rows were inserted. The variable explorer on the right shows the state of variables after execution, including `count` (225), `country` (US), `createTableCmd` (CREATE TABLE IF NOT EXISTS observations (...)), `cur` (Cursor object of sqlite3 module), `dbFile` (weather.db), `dropTableCmd` (DROP TABLE IF EXISTS observations;), `endDate` (2023-01-22T23:59:59Z), `insertCmd` (INSERT INTO observations (...)), `insertValues` (tuple of 8 values), `n` (noaa.NOAIA object), `obs` (dict), `observations` (generator object), and `start` (datetime object).

WEATHERDB FILE



QUERYING THE DATABASE

Structured Query Language (SQL) is a programming language used for working with relation databases.

SQLiteStudio was used to query the database and view the results

QUERY TO RETRIEVE ALL COLUMNS AND ALL ROWS(SS)

```

Spyder
File Edit Search Source Run Debug Consoles Projects Tools View Help
C:\Users\Tae\spyder.py\CESS110\Query.py
help window x Query.py x
1 #Purpose: Query database using SQL
2 #Name: Deontae Crenshaw
3 #Date: January 29, 2023
4 # Run BuildWeatherDB.py to build weather database before running this program
5 import sqlite3
6
7 import pandas as pd
8
9 #file names for database and output file
10 dbfile = "weather.db"
11 #format output
12 pd.set_option('display.max_rows', None)
13 pd.set_option('display.max_columns', None)
14 pd.set_option('display.width', None)
15 pd.set_option('display.max_colwidth', None)
16 pd.set_option('display.expand_from_repr', False)
17
18 #connect to and query weather database
19 conn = sqlite3.connect(dbfile)
20 #Create SQL command
21 selectCmd = "SELECT * FROM observations ORDER BY timestamp;"
22 #Print out the query
23 result = pd.read_sql_query(selectCmd, conn)
24 print(result)

```

Source Console Object

Usage

in [40]: runfile('C:/Users/Tae/spyder.py/CESS110/Query.py', wdir='C:/Users/Tae/spyder.py/CESS110')

	timestamp	windSpeed	temperature	relativeHumidity	windDirection	barometricPressure	visibility	textDescription
0	2023-01-15T16:53:00+00:00	11.16	7.2	47.249895	130.0	102440.0	16000	Clear
1	2023-01-15T17:53:00+00:00	5.40	9.4	35.825472	NaN	102340.0	16000	Clear
2	2023-01-15T18:53:00+00:00	9.36	11.7	23.866573	140.0	102200.0	16000	Clear
3	2023-01-15T19:53:00+00:00	0.00	13.9	24.481916	0.0	102100.0	16000	Clear
4	2023-01-15T20:53:00+00:00	0.00	13.9	23.383678	0.0	102070.0	16000	Clear
5	2023-01-15T21:53:00+00:00	5.40	13.9	23.383678	100.0	102030.0	16000	Clear
6	2023-01-15T22:53:00+00:00	5.40	12.2	29.740712	90.0	102000.0	16000	Clear
7	2023-01-15T23:53:00+00:00	0.00	6.1	52.890897	0.0	102030.0	16000	Clear
8	2023-01-16T00:53:00+00:00	0.00	4.4	57.340337	0.0	102000.0	16000	Clear
9	2023-01-16T01:53:00+00:00	5.40	3.3	57.059644	120.0	101930.0	16000	Clear
10	2023-01-16T02:53:00+00:00	0.00	2.6	64.188458	0.0	101970.0	16000	Clear
11	2023-01-16T03:53:00+00:00	0.00	2.6	61.374451	0.0	101930.0	16000	Clear
12	2023-01-16T04:53:00+00:00	7.56	3.9	54.699005	100.0	101970.0	16000	Clear
13	2023-01-16T05:53:00+00:00	9.36	1.1	69.293521	120.0	101930.0	16000	Clear
14	2023-01-16T06:53:00+00:00	7.56	3.9	56.799043	120.0	101930.0	16000	Clear
15	2023-01-16T07:53:00+00:00	9.36	4.4	54.634779	90.0	101830.0	16000	Clear
16	2023-01-16T08:53:00+00:00	7.56	5.6	48.581578	110.0	101800.0	16000	Clear
17	2023-01-16T09:53:00+00:00	11.16	2.6	61.374451	100.0	101830.0	16000	Clear
18	2023-01-16T10:53:00+00:00	9.36	5.6	58.439495	100.0	101830.0	16000	Clear
19	2023-01-16T11:53:00+00:00	12.96	6.1	48.725352	90.0	101860.0	16000	Mostly Cloudy
20	2023-01-16T12:53:00+00:00	7.56	6.1	50.959595	90.0	101860.0	16000	Mostly Clear
21	2023-01-16T13:53:00+00:00	11.16	7.2	49.000955	130.0	101830.0	16000	Mostly Clear
22	2023-01-16T14:53:00+00:00	9.36	6.9	47.389521	130.0	101900.0	16000	Cloudy
23	2023-01-16T15:53:00+00:00	16.56	11.7	46.482344	140.0	101830.0	16000	Mostly Cloudy
24	2023-01-16T16:53:00+00:00	20.52	15.6	43.777951	120.0	101790.0	16000	Mostly Cloudy
25	2023-01-16T17:53:00+00:00	24.12	17.8	46.245116	170.0	101520.0	16000	Partly Cloudy
26	2023-01-16T18:53:00+00:00	24.12	17.8	48.195378	200.0	101460.0	16000	Cloudy
27	2023-01-16T19:53:00+00:00	20.52	17.8	48.195378	190.0	101360.0	16000	Cloudy
28	2023-01-16T20:53:00+00:00	16.56	15.6	66.579993	190.0	101320.0	14400	Cloudy
29	2023-01-16T21:53:00+00:00	14.76	16.7	57.627586	170.0	101250.0	16000	Cloudy
30	2023-01-16T22:53:00+00:00	22.32	17.2	58.142321	180.0	101220.0	16000	Cloudy
31	2023-01-16T23:53:00+00:00	16.56	16.7	60.014618	180.0	101220.0	16000	Cloudy
32	2023-01-17T00:53:00+00:00	24.12	17.2	68.134185	180.0	101190.0	16000	Cloudy
33	2023-01-17T01:53:00+00:00	29.52	16.7	64.631043	180.0	101120.0	16000	Cloudy
34	2023-01-17T02:53:00+00:00	29.52	16.1	67.133567	180.0	101120.0	16000	Cloudy
35	2023-01-17T03:53:00+00:00	NaN	16.7	62.070091	NaN	101120.0	16000	Cloudy
36	2023-01-17T04:53:00+00:00	31.68	16.7	62.070091	190.0	101030.0	16000	Cloudy and Windy
37	2023-01-17T05:53:00+00:00	NaN	16.1	67.133567	NaN	100980.0	16000	Cloudy
38	2023-01-17T06:53:00+00:00	25.92	16.1	67.133567	190.0	100950.0	16000	Cloudy
39	2023-01-17T07:53:00+00:00	24.12	15.8	74.970824	190.0	100950.0	16000	Cloudy
40	2023-01-17T08:15:00+00:00	24.12	15.8	72.145508	190.0	100920.0	15000	Cloudy
41	2023-01-17T08:53:00+00:00	22.32	15.8	88.650223	210.0	100980.0	6400	Rain
42	2023-01-17T09:53:00+00:00	12.96	14.4	83.831769	200.0	100920.0	16000	Cloudy
43	2023-01-17T10:17:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Cloudy
44	2023-01-17T10:39:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Cloudy
45	2023-01-17T10:53:00+00:00	9.36	14.4	86.639152	190.0	100950.0	16000	Light Rain
46	2023-01-17T11:05:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Cloudy
47	2023-01-17T11:53:00+00:00	7.56	14.4	86.639152	190.0	100950.0	16000	Cloudy
48	2023-01-17T12:14:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Cloudy
49	2023-01-17T12:21:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Cloudy
50	2023-01-17T12:33:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Light Rain
51	2023-01-17T12:53:00+00:00	5.40	14.4	86.639152	130.0	100980.0	16000	Cloudy
52	2023-01-17T13:53:00+00:00	0.00	15.9	86.639136	0.0	101050.0	16000	Cloudy
53	2023-01-17T14:12:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Cloudy
54	2023-01-17T14:53:00+00:00	7.56	15.6	86.191268	170.0	101120.0	16000	Cloudy
55	2023-01-17T15:15:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Cloudy
56	2023-01-17T15:53:00+00:00	9.36	17.2	88.944625	210.0	101190.0	16000	Cloudy
57	2023-01-17T16:00+00:00	NaN	NaN	NaN	NaN	NaN	16000	Cloudy
58	2023-01-17T16:53:00+00:00	12.96	18.9	75.137346	200.0	101150.0	16000	Cloudy
59	2023-01-17T17:53:00+00:00	11.16	20.6	67.617642	200.0	101080.0	16000	Mostly Cloudy
60	2023-01-17T18:53:00+00:00	14.76	22.2	57.080138	200.0	101020.0	16000	Mostly Cloudy
61	2023-01-17T19:53:00+00:00	11.16	22.8	55.037415	NaN	100900.0	16000	Clear
62	2023-01-17T20:53:00+00:00	9.36	22.2	55.246187	200.0	101020.0	16000	Partly Cloudy
63	2023-01-17T21:53:00+00:00	9.36	22.8	51.212777	240.0	101050.0	16000	Mostly Clear
64	2023-01-17T22:53:00+00:00	9.36	20.6	52.530352	210.0	101000.0	16000	Mostly Clear
65	2023-01-17T23:53:00+00:00	7.56	17.2	67.362333	200.0	101220.0	15000	Mostly Cloudy
66	2023-01-18T00:53:00+00:00	0.00	16.1	77.679888	0.0	101290.0	16000	Clear
67	2023-01-18T01:53:00+00:00	5.40	15.6	88.282282	140.0	101530.0	10000	Clear
68	2023-01-18T02:53:00+00:00	7.56	12.2	86.423197	140.0	101500.0	11270	Clear

Python Console History Terminal

File Edit Search Source Run Debug Consoles Projects Tools View Help

Python 3.8.10

7:00 PM 1/29/2023

QUERY TO RETRIEVE LOWEST AND HIGHEST TEMPERATURES (SCREENSHOT)

The screenshot shows the Spyder Python IDE interface. The left pane displays a Python script named 'Query.py' with the following code:

```
1 #Purpose: Query database using SQL.
2
3 #Name: Deontae Crenshaw
4
5 #Date: January 29, 2023
6
7 # Run BuildWeatherDB.py to build weather database before running this program
8
9 import sqlite3
10
11 import pandas as pd
12
13 #file names for database and output file
14
15 dbFile = "weather.db"
16
17 #format output
18
19 pd.set_option('display.max_rows', None)
20
21 pd.set_option('display.max_columns', None)
22
23 pd.set_option('display.width', None)
24
25 pd.set_option('display.max_colwidth', None)
26
27 pd.set_option('display.expand_frame_repr', False)
28
29 #connect to and query weather database
30
31 conn = sqlite3.connect(dbFile)
32
33 #Create SQL command
34
35 selectCmd = "SELECT min(temperature), max(temperature) FROM observations;"
36
37 #print out the query
38
39 result = pd.read_sql_query(selectCmd, conn)
40
41 print(result)
```

The right pane shows the IPython console with the following output:

```
In [11]: runfile('C:/Users/Tae/.spyder-py3/CEISS110/Query.py', wdir='C:/Users/Tae/.spyder-py3/CEISS110')
min(temperature) max(temperature)
0                1.1             22.8

In [12]:
```

The bottom status bar indicates the Python version is 3.8.10 and the file encoding is UTF-8.

QUERY TO RETRIEVE ALL CLEAR DAYS (SCREENSHOT)

The screenshot shows the Spyder Python IDE interface. The left pane contains a Python script named 'Query.py' with the following code:

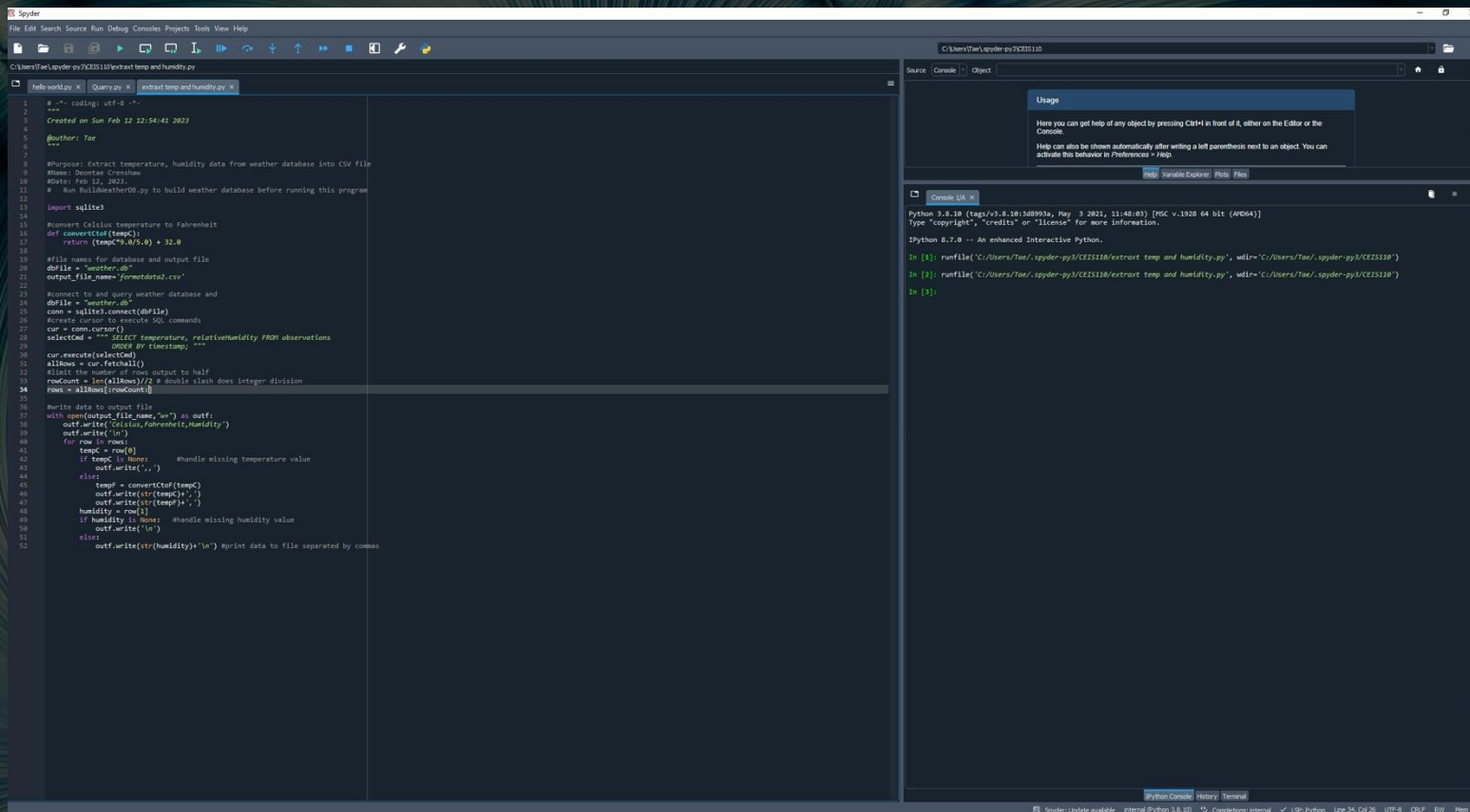
```
1 #Purpose: Query database using SQL
2
3 #Name: Desmitae Crenshaw
4
5 #Date: January 29, 2023
6
7 # Run BuildWeatherDB.py to build weather database before running this program
8
9 import sqlite3
10
11 import pandas as pd
12
13 #file names for database and output file
14 dbFile = "weather.db"
15
16 #format output
17
18 pd.set_option('display_max_rows', None)
19
20 pd.set_option('display_max_columns', None)
21
22 pd.set_option('display_width', None)
23
24 pd.set_option('display_max_colwidth', None)
25
26 pd.set_option('display_expand_frame_repr', False)
27
28 #connect to and query weather database
29
30 conn = sqlite3.connect(dbFile)
31
32 #Create SQL command
33
34 selectCmd = " SELECT temperature, windSpeed, testDescription FROM observations where testDescription = 'Clear'; "
35
36 #print out the query
37
38 result = pd.read_sql_query(selectCmd, conn)
39
40 print(result)
```

The right pane shows the 'Console' tab with the output of the script, which is a table of weather data:

```
In [9]: runfile('C:/Users/Tan/.spyder-py3/CEIS118/Query.py', wdir='C:/Users/Tan/.spyder-py3/CEIS118')
temperature windSpeed testDescription
0 10.6 5.40 Clear
1 11.7 7.56 Clear
2 12.2 6.00 Clear
3 12.8 11.16 Clear
4 11.7 5.40 Clear
5 10.6 5.40 Clear
6 9.4 11.16 Clear
7 6.1 9.36 Clear
8 5.6 9.36 Clear
9 3.3 7.56 Clear
10 1.7 6.00 Clear
11 2.8 7.56 Clear
12 2.8 9.36 Clear
13 3.3 7.56 Clear
14 3.9 11.16 Clear
15 3.3 14.76 Clear
16 4.4 7.56 Clear
17 5.6 11.16 Clear
18 5.6 11.16 Clear
19 5.6 5.40 Clear
20 5.6 7.56 Clear
21 6.7 5.40 Clear
22 8.9 9.36 Clear
23 10.6 5.40 Clear
24 12.2 9.36 Clear
25 12.2 10.56 Clear
26 11.7 10.56 Clear
27 10.6 11.16 Clear
28 10.6 12.96 Clear
29 9.4 9.36 Clear
30 6.7 10.56 Clear
31 5.6 9.36 Clear
32 1.7 11.16 Clear
33 3.9 9.36 Clear
```

The bottom status bar shows the Python version (3.8.10) and the file path (C:/Users/Tan/.spyder-py3/CEIS118/Query.py).

EXTRACTING TEMPERATURE AND HUMIDITY USING PYTHON CODE



The screenshot displays the Spyder Python IDE interface. The main editor window shows a Python script named 'extract temp and humidity.py'. The script's purpose is to extract temperature and humidity data from a SQLite database and save it to a CSV file. It includes comments in Chinese and English, a docstring, and a series of code blocks for database connection, data retrieval, and file output.

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Sun Feb 12 12:54:41 2023
4 @author: Tao
5 """
6
7 #Purpose: Extract temperature, humidity data from weather database into CSV file
8 #Name: Dantao Chen
9 #Date: Feb 12, 2023.
10 # Run BuildWeatherDB.py to build weather database before running this program
11
12 import sqlite3
13
14 #convert Celsius temperature to Fahrenheit
15 def convertCtoF(tempC):
16     return (tempC*9.0/5.0) + 32.0
17
18 #file names for database and output file
19 dbfile = "weather.db"
20 output_file_name = "forecastdata2.csv"
21
22 #connect to and query weather database and
23 dbfile = "weather.db"
24 conn = sqlite3.connect(dbfile)
25 #create cursor to execute SQL commands
26 cur = conn.cursor()
27
28 selectCmd = """ SELECT temperature, relativeHumidity FROM observations
29 ORDER BY timestamp; """
30 cur.execute(selectCmd)
31 allRows = cur.fetchall()
32 #limit the number of rows output to half
33 rowCount = len(allRows)//2 # double slash does integer division
34 rows = allRows[rowCount:]
35
36 #write data to output file
37 with open(output_file_name, "w") as outf:
38     outf.write("Celsius,Fahrenheit,humidity")
39     outf.write("\n")
40     for row in rows:
41         temp = row[0]
42         if tempC is None: #handle missing temperature value
43             outf.write(",")
44         else:
45             temp = convertCtoF(tempC)
46             outf.write(str(temp)+",")
47             outf.write(str(temp)+",")
48         humidity = row[1]
49         if humidity is None: #handle missing humidity value
50             outf.write("\n")
51         else:
52             outf.write(str(humidity)+",\n") #print data to file separated by commas
```

The right-hand pane of the IDE shows the 'Console' window with the following output:

```
Python 3.8.10 (tags/v3.8.10:3b89933a, May 3 2021, 11:48:03) [MSC v.1928 64 bit (AMD64)]
Type "copyright", "credits" or "license()" for more information.
Python 3.8.10 -- An enhanced interactive Python.

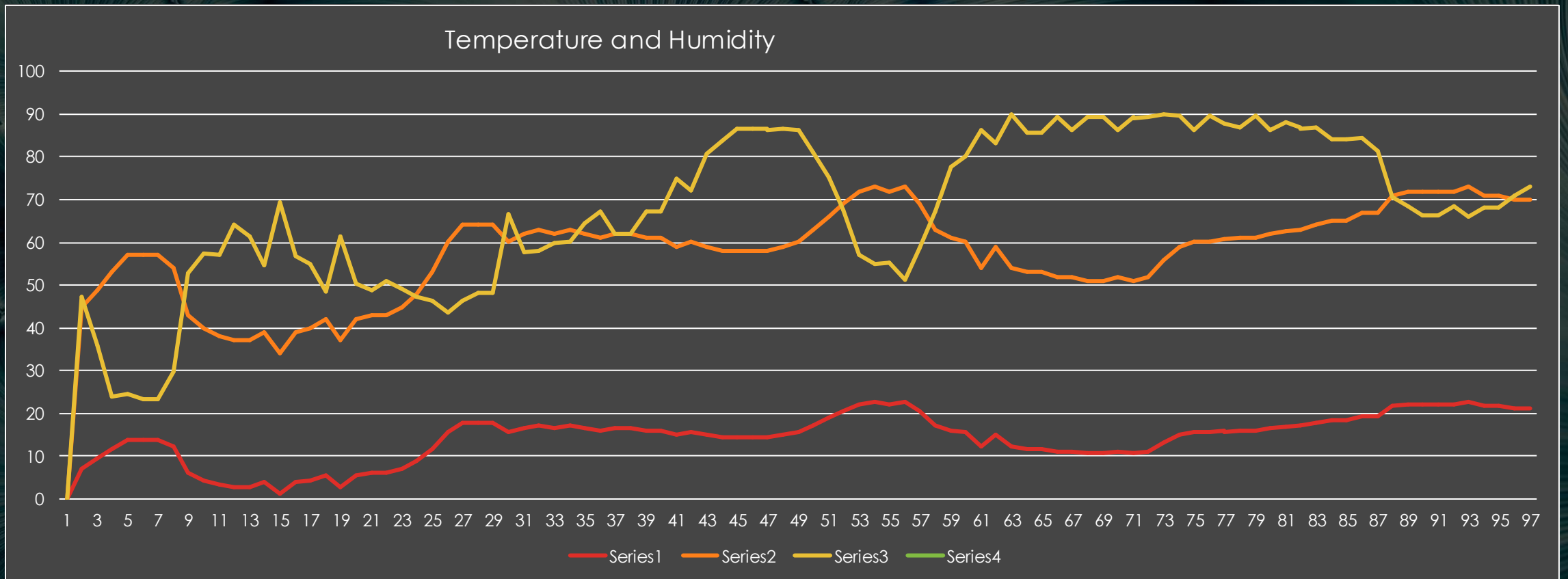
In [1]: runfile('C:/Users/Tao/.spyder-py3/CEIS110/extract temp and humidity.py', wdir='C:/Users/Tao/.spyder-py3/CEIS110')
In [2]: runfile('C:/Users/Tao/.spyder-py3/CEIS110/extract temp and humidity.py', wdir='C:/Users/Tao/.spyder-py3/CEIS110')
In [3]:
```

The status bar at the bottom indicates the current file is 'extract temp and humidity.py', the Python version is 3.8.10, and the file encoding is UTF-8.

DATA FORMATTED IN AN EXCEL SPREADSHEET

1	Celsius	Fahrenheit	Humidity	
2	7.2	44.96	47.2499	
3	9.4	48.92	35.82567	
4	11.7	53.06	23.86657	
5	13.9	57.02	24.48192	
6	13.9	57.02	23.38367	
7	13.9	57.02	23.38367	
8	12.2	53.96	29.74071	
9	6.1	42.98	52.8901	
10	4.4	39.92	57.34894	
11	3.3	37.94	57.05964	
12	2.8	37.04	64.18845	
13	2.8	37.04	61.37445	
14	3.9	39.02	54.69907	
15	1.1	33.98	69.29152	
16	3.9	39.02	56.79094	
17	4.4	39.92	54.83478	
18	5.6	42.08	48.58157	
19	2.8	37.04	61.37445	
20	5.6	42.08	50.43949	
21	6.1	42.98	48.72535	
22	6.1	42.98	50.95939	
23	7.2	44.96	49.04006	
24	8.9	48.02	47.38952	
25	11.7	53.06	46.48234	
26	15.6	60.08	43.73795	
27	17.8	64.04	46.24512	
28	17.8	64.04	48.19538	
29	17.8	64.04	48.19538	
30	15.6	60.08	66.57999	
31	16.7	62.06	57.62751	
32	17.2	62.96	58.14282	
33	16.7	62.06	60.01462	
34	17.2	62.96	60.13419	
35	16.7	62.06	64.61844	
36	16.1	60.98	67.13357	
37	16.7	62.06	62.07009	
38	16.7	62.06	62.07009	
39	16.1	60.98	67.13357	
40	16.1	60.98	67.13357	
41	15	59	74.97803	
42	15.6	60.08	72.14551	
43	15	59	80.65022	
44	14.4	57.92	83.8317	
45	14.4	57.92	86.63915	
46	14.4	57.92	86.63915	
47	14.4	57.92	86.63915	
48	15	59	86.69714	
49	15.6	60.08	86.19127	
50	17.2	62.96	80.94463	
51	18.9	66.02	75.13737	
52	20.6	69.08	67.61764	
53	22.2	71.96	57.08013	
54	22.8	73.04	55.03702	

DATA VISUALIZATION



HISTOGRAM OF HUMIDITY

#Purpose: Create a histogram of humidity data from the second period

#Name: Deontae Crenshaw

#Date: Feb 18, 2023

```
import pandas as pd
```

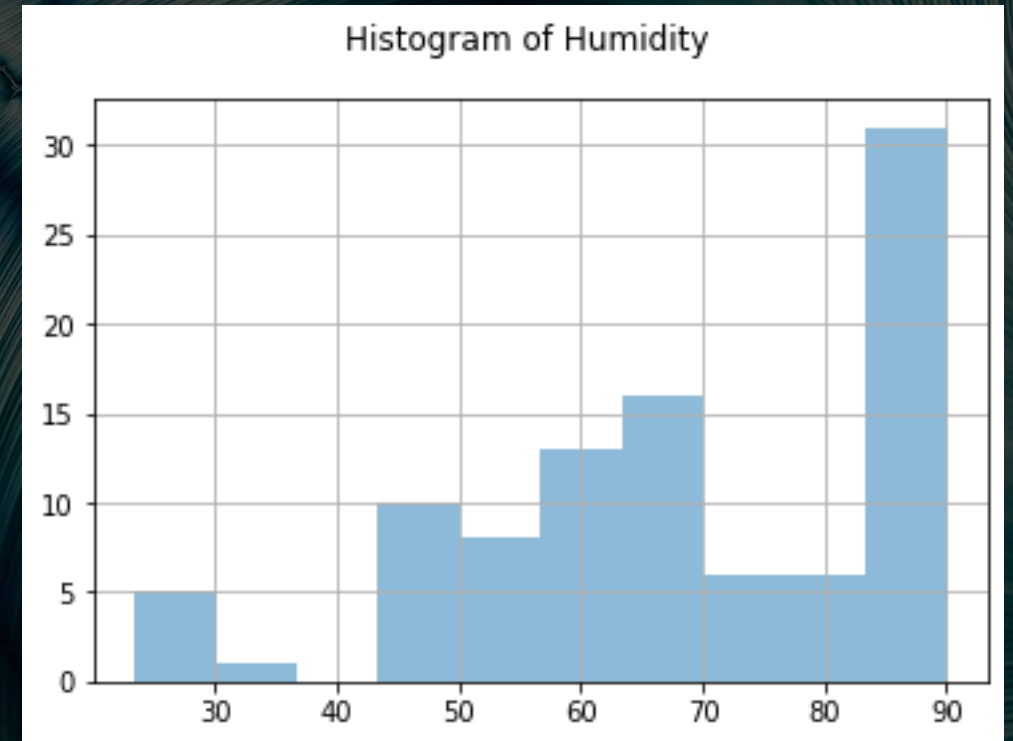
```
import matplotlib.pyplot as plt
```

```
df1 = pd.read_csv("formatdata.csv")
```

```
df2 = pd.read_csv("formatdata2.csv")
```

```
df2['Humidity'].hist(bins=10, alpha=0.5);  
plt.suptitle('Histogram of Humidity')
```

```
plt.show()
```



PERIOD 2 BOX PLOT

```
#Purpose: Create box plot for period 2 data
#Name: Deontae Crenshaw
#Date: Feb 12, 2023
import pandas as pd
import matplotlib.pyplot as plt
df2 = pd.read_csv("formatdata2.csv")
df2.boxplot(); plt.suptitle('Period 2 box plot')

print(df2.info())
print(df2.describe())
```

